



Proyecto de Investigación Sistemas Operativos II

Diseño e implementación de un sistema distribuido de monitoreo de métricas del sistema operativo con detección de anomalías mediante aprendizaje automático

Curso:

BISOFT - 34 - Sistemas Operativos II

Carrera:

Ingeniería del Software

Profesor:

Carlos Andrés Mendéz Rodríguez

Estudiante:

Santiago Osejo Lobo

Institución:

Universidad Latina de Costa Rica

II Cuatrimestre 2026

Tema de investigación

El presente proyecto se enmarca dentro del tema asignado correspondiente al monitor inteligente de sistema con detección de anomalías, y consiste en el diseño e implementación de un sistema distribuido de monitoreo de métricas del sistema operativo, capaz de identificar comportamientos anómalos mediante un modelo de aprendizaje automático no supervisado. El interés por este tema surge de la necesidad que tienen las infraestructuras actuales, tanto en entornos locales como en la nube, de contar con mecanismos de observabilidad que permitan anticipar fallos antes de que afecten la disponibilidad del servicio. A diferencia de los enfoques tradicionales basados en umbrales fijos, que suelen generar falsos positivos ante cargas de trabajo variables, este proyecto explora un enfoque basado en el algoritmo Isolation Forest para detectar picos de uso irregulares o procesos sospechosos, a partir de la lectura directa de métricas del sistema operativo como el uso de CPU, memoria, procesos y tráfico de red. Su desarrollo permitirá analizar, de forma práctica, los criterios de alta disponibilidad, rendimiento, escalabilidad y seguridad abordados en el curso de Sistemas Operativos II, mediante una arquitectura cliente-servidor desplegada en contenedores Docker.

Objetivos

Objetivo General

Diseñar e implementar un sistema cliente-servidor de monitoreo remoto de métricas del sistema operativo, capaz de detectar comportamientos anómalos mediante un modelo de aprendizaje automático no supervisado, evaluando su desempeño en términos de disponibilidad, escalabilidad, seguridad y tiempo de respuesta.

Objetivos Específicos

1. Analizar los criterios de alta disponibilidad, rendimiento, escalabilidad y seguridad aplicables a sistemas de monitoreo distribuido en entornos de red y nube.
2. Diseñar una arquitectura cliente-servidor para la recolección remota de métricas del sistema operativo (procesos, CPU, memoria y red), a partir de la lectura de /proc.
3. Implementar un módulo de detección de anomalías basado en un modelo de aprendizaje automático no supervisado (Isolation Forest), capaz de identificar picos de uso irregulares o procesos sospechosos.
4. Establecer métricas de evaluación de rendimiento —tiempo de respuesta, distribución del tráfico y tiempo de inactividad— para validar el comportamiento del sistema bajo distintas condiciones de carga.
5. Proponer mejoras al diseño e implementación del sistema con base en los resultados obtenidos durante las pruebas.

Metodología

Tipo y enfoque de la investigación

El proyecto se ubica dentro de la investigación aplicada, con un enfoque cuantitativo y un diseño de tipo experimental. A diferencia de un trabajo puramente documental, aquí se construye un artefacto de software —un prototipo de sistema cliente-servidor— y se somete a pruebas controladas para poder responder a los objetivos específicos relacionados con el rendimiento, la disponibilidad y la capacidad de detección del modelo. El diseño experimental permite variar condiciones, como el número de clientes concurrentes o la tasa de anomalías inyectadas en los

datos, y observar su efecto sobre variables medibles: tiempo de respuesta, throughput, exactitud del modelo y tiempo de inactividad ante una caída del servidor.

El proceso de trabajo se organiza en cinco fases, que corresponden a las últimas cinco semanas del cronograma del curso: (1) revisión de literatura académica sobre observabilidad, sistemas distribuidos y detección de anomalías; (2) diseño de la arquitectura cliente-servidor; (3) implementación del prototipo en Python; (4) ejecución de experimentos controlados sobre el prototipo; y (5) análisis de los resultados obtenidos y propuesta de mejoras, en línea con el objetivo específico 5 planteado en la sección anterior.

Plan de trabajo

A continuación se detalla el plan de trabajo para las semanas restantes del proyecto:

Semana	Actividades	Entregable
Semana 12	Definición de la arquitectura cliente-servidor, selección de tecnologías (Python, sockets TCP, scikit-learn, Docker) y elaboración del plan de trabajo y la metodología.	Documento de planificación y metodología
Semana 13	Búsqueda y análisis de literatura académica sobre observabilidad de sistemas distribuidos, contenedores y detección de anomalías con aprendizaje automático no supervisado; redacción del marco teórico.	Marco teórico documentado con referencias APA
Semana 14	Implementación del prototipo cliente-servidor, entrenamiento del modelo Isolation Forest, ejecución de pruebas de concurrencia, latencia y tolerancia a fallos, análisis de los resultados obtenidos.	Prototipo funcional e informe de resultados
Semana 15	Empaquetado del sistema en Docker, documentación del repositorio en GitHub (README, licencia open source), preparación de la demo y la presentación final.	Repositorio público, informe final y presentación

Herramientas y recursos

Para la implementación se seleccionó Python 3, dado su ecosistema maduro de librerías para aprendizaje automático (scikit-learn) y su soporte nativo para comunicación en red mediante el módulo socket, sin necesidad de frameworks adicionales. La concurrencia se maneja con threading, lo cual resulta suficiente para la escala de este prototipo (decenas de clientes simultáneos); para un despliegue de mayor escala se consideraría un modelo asíncrono (asyncio) o basado en múltiples procesos.

El modelo de detección de anomalías es Isolation Forest, implementado mediante scikit-learn. Para la lectura de métricas del sistema operativo se emplea el sistema de archivos /proc de Linux (/proc/stat, /proc/meminfo, /proc/net/dev), conforme lo exige el objetivo específico 2. En la fase de experimentación controlada, descrita en la sección de Desarrollo e implementación, se utilizan métricas generadas sintéticamente a partir de distribuciones calibradas según el comportamiento típico de un servidor; esto permite conocer de antemano cuáles muestras son anómalas y medir objetivamente el desempeño del modelo, algo que no sería posible con datos reales sin etiquetar.

Como recursos adicionales se contempla Docker para el empaquetado y despliegue del sistema, y un repositorio en GitHub con licencia open source y archivo README, tal como lo solicita el enunciado del proyecto para la entrega final.

Marco teórico

Sistemas distribuidos y monitoreo remoto

Un sistema distribuido puede entenderse como un conjunto de componentes autónomos que se presentan ante sus usuarios como un sistema único y coherente (Tanenbaum & Van Steen, 2017). Esta definición es central para el proyecto, ya que el sistema de monitoreo propuesto está compuesto por al menos dos entidades independientes —el agente que recolecta métricas y el servidor que las analiza— que deben coordinarse a través de la red para cumplir un objetivo común. Tanenbaum y Van Steen (2017) señalan que la escalabilidad, la tolerancia a fallos y la transparencia frente al usuario son atributos deseables en este tipo de arquitecturas; estos tres criterios se retoman más adelante como base para diseñar las pruebas de carga y de caída del prototipo, y se relacionan directamente con el objetivo específico 1 de este proyecto.

Observabilidad en sistemas distribuidos y contenedores

En la literatura reciente sobre sistemas distribuidos se distingue entre monitoreo, entendido como la supervisión y el registro del comportamiento de un sistema, y observabilidad, un concepto tomado de la teoría de control que mide el grado en que el estado interno de un sistema puede inferirse a partir de sus salidas —típicamente métricas, registros (logs) y trazas (traces)— (Niedermaier et al., 2019). Niedermaier et al. (2019) realizaron un estudio cualitativo con profesionales de la industria y encontraron que la creciente complejidad de las arquitecturas de microservicios exige una estrategia organizacional explícita para la observabilidad, más allá de instrumentar herramientas de forma aislada.

Por su parte, Usman et al. (2022) realizaron una revisión sistemática de la observabilidad en entornos distribuidos de borde (edge) y microservicios contenedorizados, identificando que la

disponibilidad de telemetría —como latencia, tráfico, errores y saturación de recursos— es un requisito común para poder detectar y diagnosticar fallos antes de que afecten a los usuarios finales. Esta clasificación de señales coincide con las métricas que este proyecto recolecta (CPU, memoria, procesos y red) y respalda la selección de estas cuatro variables como base del vector de características que se entrega al modelo de detección de anomalías.

En cuanto al entorno de despliegue, Felter et al. (2015) compararon el rendimiento de máquinas virtuales y contenedores Linux (usando Docker) y encontraron que los contenedores introducen una sobrecarga de CPU, memoria y red considerablemente menor que la virtualización tradicional. Este hallazgo respalda la decisión, planteada en el enunciado del proyecto, de utilizar Docker como mecanismo de despliegue: al introducir poca sobrecarga adicional, no debería distorsionar de forma significativa las métricas de rendimiento que se busca medir con el prototipo.

Detección de anomalías mediante aprendizaje automático no supervisado

El algoritmo Isolation Forest, propuesto originalmente por Liu et al. (2008), parte de un principio distinto al de la mayoría de los métodos de detección de anomalías previos: en lugar de construir un perfil de lo que es normal y señalar como anómalo lo que no encaja en dicho perfil, el algoritmo aísla directamente las observaciones atípicas. Esto se logra construyendo árboles de partición aleatoria; dado que las anomalías son, por definición, pocas y distintas del resto de los datos, tienden a quedar aisladas en particiones más pequeñas y con menos divisiones que las observaciones normales. Los propios autores destacan que este enfoque tiene una complejidad temporal lineal y un requerimiento de memoria bajo, lo cual lo hace apropiado para escenarios de monitoreo en tiempo real como el de este proyecto.

La aplicación de Isolation Forest específicamente a métricas de infraestructuras en la nube ha sido documentada en trabajos recientes. Mitropoulou et al. (2024) lo emplearon, junto con el algoritmo CBLOF, para detectar eventos de sobreuso de cómputo y almacenamiento en infraestructuras de nube representadas como grafos de conocimiento, evaluando su desempeño en un entorno simulado. Este antecedente es relevante para el presente proyecto porque valida el uso de un entorno simulado y de datos generados de forma controlada como estrategia legítima para evaluar cuantitativamente un detector de anomalías no supervisado, antes de desplegarlo sobre datos reales de producción.

Síntesis del marco teórico

En conjunto, la literatura revisada permite ubicar este proyecto en la intersección de dos campos: la observabilidad de sistemas distribuidos, que define qué señales recolectar y por qué, y la detección de anomalías con aprendizaje automático no supervisado, que define cómo interpretar esas señales sin necesidad de contar con datos previamente etiquetados. Sobre esta base teórica se diseñó la arquitectura del prototipo y el conjunto de pruebas que se describen en la siguiente sección.

Desarrollo e implementación

Arquitectura del prototipo

El prototipo implementa una arquitectura cliente-servidor sobre sockets TCP, comunicándose mediante mensajes en formato JSON. Cada cliente representa un nodo monitoreado que envía periódicamente una muestra de métricas (uso de CPU, uso de memoria, cantidad de procesos activos y tráfico de red) al servidor central. El servidor mantiene en memoria un modelo Isolation Forest previamente entrenado; al recibir una muestra, la evalúa y responde al

cliente con un veredicto (normal o anómala) y un puntaje de anomalía, todo dentro de la misma conexión TCP.

La concurrencia del lado del servidor se maneja atendiendo cada conexión entrante en un hilo independiente (threading), lo que permite recibir y evaluar métricas de varios clientes de forma simultánea sin bloquear al resto. Para la lectura de métricas reales del sistema operativo se implementó un módulo independiente que consulta directamente `/proc/stat` (uso de CPU calculado a partir de dos lecturas consecutivas del tiempo de CPU ocioso), `/proc/meminfo` (uso de memoria), el listado numérico de `/proc` (cantidad de procesos activos) y `/proc/net/dev` (tráfico acumulado de red). Una ejecución de prueba de este módulo, sobre el entorno de desarrollo, produjo la siguiente salida real:

```
CPU uso: 0.0 %  
Memoria uso: 7.01 % (total=4093756 kB, disponible=3806820 kB)  
Procesos activos: 51  
Bytes de red acumulados (rx+tx, todas las interfaces): 1417450
```

Esta lectura confirma que el módulo obtiene valores reales del sistema operativo subyacente, cumpliendo el objetivo específico 2. Sin embargo, para el experimento controlado que se describe a continuación se optó por generar métricas de forma sintética en lugar de usar directamente esta lectura en vivo: al no existir, en el entorno de pruebas, procesos anómalos reales que generar de forma segura y repetible, no habría manera de conocer con certeza qué muestras son realmente anómalas y así poder calcular métricas de precisión del modelo. La generación sintética, calibrada con distribuciones representativas de un servidor típico, resuelve este problema sin sacrificar el resto de la arquitectura, que sí se ejerce de forma completa (sockets, concurrencia, formato de mensajes y evaluación con el modelo).

Diseño experimental

El experimento se dividió en dos partes. En la primera, el servidor se entrenó con un lote inicial (bootstrap) de 600 muestras consideradas normales, usando un Isolation Forest de 150 árboles y una contaminación esperada de 5 %. Posteriormente se generó un flujo de 400 métricas adicionales, con una tasa real de anomalías del 5 % (20 muestras anómalas de tipo pico de CPU, fuga de memoria, explosión de procesos o tráfico de red anómalo, y 380 muestras normales), repartidas entre 20 clientes concurrentes que enviaron sus métricas en oleadas simultáneas. Se midió el tiempo de respuesta de cada petición, el throughput global del servidor y se comparó cada predicción del modelo contra la etiqueta real de la muestra (que el cliente conocía, pero que en ningún momento se le entregó al modelo).

En la segunda parte se evaluó la tolerancia a fallos: se detuvo el servidor a la mitad de una tanda de peticiones y se midió cuánto tardaba un cliente en detectar la caída y cuántos intentos de reconexión fallidos ocurrían antes de ello, como una aproximación al tiempo de inactividad percibido por un cliente del sistema.

Resultados

La tabla 1 resume los resultados obtenidos en el experimento de concurrencia y detección:

Métrica	Valor obtenido
Peticiones totales enviadas	400
Peticiones exitosas / fallidas	400 / 0
Cientes concurrentes	20
Duración total del experimento	7.90 s
Throughput	50.66 peticiones/s
Latencia promedio	229.27 ms

Métrica	Valor obtenido
Latencia mediana	226.99 ms
Latencia p95	380.23 ms
Latencia máxima	415.72 ms
Exactitud (accuracy)	0.9575
Precisión	0.56
Exhaustividad (recall)	0.70
F1-score	0.6222

La matriz de confusión (figura 1), sobre las 400 muestras evaluadas (20 realmente anómalas y 380 normales), muestra que el modelo identificó correctamente 14 de las 20 anomalías reales (recall de 70 %), a costa de 11 falsas alarmas sobre muestras normales (precisión de 56 %):

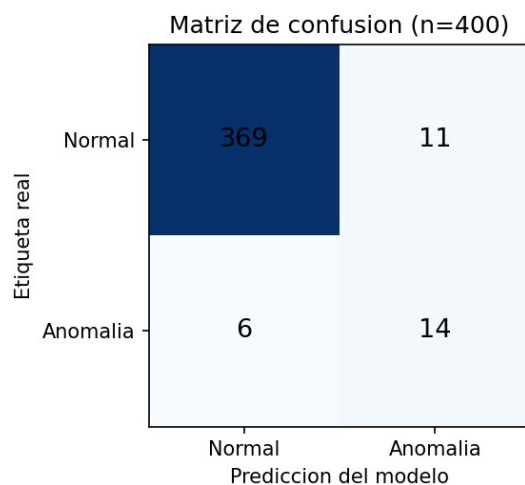


Figura 1. Matriz de confusión del modelo Isolation Forest sobre el flujo de prueba ($n = 400$).

En la prueba de tolerancia a fallos, el cliente detectó la caída del servidor tras 6 intentos de reconexión fallidos, en un tiempo total de aproximadamente 2.99 segundos, usando un tiempo de espera (timeout) de conexión de 2 segundos por intento más los reintentos espaciados cada 0.3 segundos configurados en el cliente de prueba.

Análisis de resultados y ajustes necesarios

La exactitud general (95.75 %) resulta engañosamente alta porque la mayoría de las muestras son normales; las métricas más informativas para este caso de uso son la precisión y el recall. Un recall de 70 % indica que el modelo, entrenado únicamente con un lote inicial de comportamiento normal y sin ajuste fino de hiperparámetros, logra capturar la mayoría de las anomalías inyectadas, lo cual es razonable para un primer prototipo. La precisión de 56 % implica que un poco menos de la mitad de las alertas generadas son falsas alarmas; en un sistema de monitoreo real esto podría ajustarse reduciendo el parámetro de contaminación esperada o incorporando una ventana de confirmación (varias lecturas consecutivas anómalas antes de generar una alerta) en lugar de decidir con una sola muestra, como se hizo en este prototipo.

Respecto al tiempo de respuesta, un promedio de 229 ms por petición es alto para un sistema de monitoreo que debería idealmente reportar en tiempo casi real. La causa principal identificada es que cada métrica se envía abriendo una nueva conexión TCP, lo que añade el costo del saludo de tres vías (three-way handshake) a cada petición y genera contención cuando 20 hilos cliente compiten por establecer conexión casi al mismo tiempo. Una mejora directa, que se propone como trabajo futuro en línea con el objetivo específico 5, es mantener una conexión persistente por cliente en lugar de abrir una nueva por cada métrica enviada, o adoptar un protocolo asíncrono (asynco) del lado del servidor para reducir la contención entre hilos.

Finalmente, el tiempo de aproximadamente 3 segundos para detectar la caída del servidor está determinado directamente por los parámetros de timeout configurados en el cliente; un sistema productivo debería ajustar estos valores según el nivel de servicio (SLA) requerido, y complementarse con un mecanismo de heartbeat o con réplicas del servidor para reducir el tiempo de inactividad real percibido por los nodos monitoreados.

Conclusiones

El desarrollo de este proyecto permitió cubrir, de forma práctica, los cinco objetivos específicos planteados al inicio. Respecto al primero, la revisión de literatura sobre sistemas distribuidos y observabilidad (Tanenbaum & Van Steen, 2017; Niedermaier et al., 2019; Usman et al., 2022) permitió identificar la escalabilidad, la tolerancia a fallos y la disponibilidad de telemetría como criterios centrales, que luego se tradujeron en pruebas concretas sobre el prototipo. El segundo objetivo se cumplió mediante una arquitectura cliente-servidor sobre sockets TCP capaz de leer métricas reales desde /proc, aun cuando el experimento cuantitativo final se realizó sobre datos sintéticos por razones metodológicas explicadas en la sección anterior.

El tercer objetivo, referido a la implementación del módulo de detección de anomalías, se alcanzó mediante un modelo Isolation Forest que logró un recall de 70 % y una precisión de 56 % sobre el flujo de prueba, resultados razonables para un prototipo sin ajuste fino de hiperparámetros. El cuarto objetivo se cubrió estableciendo y midiendo explícitamente tiempo de respuesta, throughput y tiempo de inactividad ante una caída del servidor, lo que a su vez permitió identificar limitaciones concretas del diseño actual —principalmente el uso de conexiones TCP no persistentes— y así dar cumplimiento al quinto objetivo, proponiendo mejoras específicas como el uso de conexiones persistentes, un modelo asíncrono del lado del servidor y un mecanismo de confirmación por ventana antes de emitir una alerta.

Como limitación principal debe señalarse que los resultados cuantitativos de detección se obtuvieron sobre un flujo de métricas sintéticas y no sobre tráfico real de producción; esto fue una decisión metodológica deliberada para poder contar con etiquetas de verdad conocida y así evaluar objetivamente al modelo, pero implica que el desempeño reportado debe interpretarse como una

cota orientativa y no como una medición definitiva del comportamiento del sistema en un entorno real. Como trabajo pendiente para las siguientes entregas queda el empaquetado del sistema en contenedores Docker, la publicación del repositorio en GitHub con su respectiva licencia y documentación (README), la incorporación de cifrado TLS en la comunicación entre cliente y servidor, y la validación del prototipo con lecturas reales y continuas de /proc en lugar de datos sintéticos.

Referencias

- Felter, W., Ferreira, A., Rajamony, R., & Rubio, J. (2015). An updated performance comparison of virtual machines and Linux containers. *2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 171–172. <https://doi.org/10.1109/ISPASS.2015.7095802>
- Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining*, 413–422. <https://doi.org/10.1109/ICDM.2008.17>
- Mitropoulou, K., Kokkinos, P., Soumplis, P., & Varvarigos, E. (2024). Anomaly detection in cloud computing using knowledge graph embedding and machine learning mechanisms. *Journal of Grid Computing*, 22(1), Artículo 6. <https://doi.org/10.1007/s10723-023-09727-1>
- Niedermaier, S., Koetter, F., Freymann, A., & Wagner, S. (2019). On observability and monitoring of distributed systems: An industry interview study. *arXiv*. <https://arxiv.org/abs/1907.12240>

Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed systems: Principles and paradigms* (3rd ed.). Distributed-systems.net.

Usman, M., Ferlin, S., Brunstrom, A., & Taheri, J. (2022). A survey on observability of distributed edge & container-based microservices. *IEEE Access*, 10, 86904–86919.
<https://doi.org/10.1109/ACCESS.2022.3193102>